Debugging
Assignment 3

Deadline: June 01, 2026 17:15

- Submissions must be made through **Moodle** as a single **ZIP archive (.zip)**. Submissions by Email will not be accepted.
- The archive must contain: source code, answers and explanations for other questions.
- Any solution that is not source code must be submitted as **PDF files**. If you write by hand, submit a high-quality scan/photo combined into a single PDF; ensure your solution is **readable** and **well-organized**.
- You may ask questions via **Discord** in `db-exercise`, but do *not* post solutions or code there. Conceptual questions only.
- The submission will be manually graded. Results will be published approximately 1–2 weeks after the deadline.
- Do not share solutions publicly; cite any external sources; each solution must be your own work.

Task 1: Delta Debugger for LaTeX files (10 points)

`LaTeX` is a document preparation system widely used for creating technical and scientific documentation. It allows users to write structured documents using plain text and commands. LaTeX files can be compiled into, e.g., PDF files. However, sometimes this compilation fails due to syntax errors, missing packages, or other issues. Error messages can sometimes be hard to decipher. Thus, debugging such errors can become very time-consuming, especially for large documents.

In this exercise, you will implement a delta debugger in a programming language of your choice. The goal of this delta debugger is to find the smallest possible failing subset of a broken LaTeX file that still reproduces the same error as the initial file.

Problem Statement

You are provided with some broken LaTeX files each reporting an error during compilation with *pdflatex*. Using the files and *pdflatex*, for each broken LaTeX file, your implementation should:

1. Compile the LaTeX file with *pdflatex* and save the produced error (It should be the same error in the end).
2. Parse the broken LaTeX file.
3. Reduce the LaTeX file iteratively using delta debugging, while still preserving the same error throughout.
4. Provide a minimized version of the broken LaTeX file.

You may implement the delta debugger in a line-by-line manner. For example, you can iteratively minimize the file by removing lines. The delta debugger does not need to be agnostic to LaTeX syntax.

If there are multiple errors in a LaTeX file, the delta debugger only needs to preserve the first error reported by *pdflatex*.

Deliverables

- Well-documented source code of your delta debugger implementation.
- Build and run instructions for your implementation.
- All the minimized broken LaTeX files.

Hints

- You will likely need to parse the LaTeX file upfront, as LaTeX documents rely on certain structural commands to compile successfully. When removed during delta debugging this likely results in a different error. These structural commands include:
  - $\backslash documentclass\{...\}$
  - $\backslash begin\{document\}$ and $\backslash end\{document\}$
- Keep track of the error message after each reduction, it should stay the same:
  - Pay attention to changes in line numbers. To preserve error locations, you may need to replace removed lines with empty lines rather than deleting them entirely initially.
  - If you encounter an error that is different to the initial error you will need to backtrack.

Task 2: Property-Based Testing with Hypothesis (10 points)

1. **Properties for Property-Based Debugging** (4 points)

   You are given implementations of algorithms in *simple_functions.py*. The goal is to check whether those implementations are bug free using `hypothesis`.

   For each of the given functions:

   - Write at least 2 properties that help you identify any bugs.
   - Report whether the implementation is bug free, and briefly explain why based on your properties.
   - Run `hypothesis`. If a property fails, report the minimal failing example.
   - If the implementation contains a bug, find and correct it, and check if your properties hold now.

   The file *simple_functions.py* contains the following functions:

   - `maximum(l: list[int]) -> int`: Returns the largest element of a list of integers.
   - `insertion_sort(l: list[int]) -> list[int]`: Sorts a list of integers in ascending order.
   - `gcd(a: int, b: int) -> int`: Returns the greatest common divisor (GCD) of two integers.
   - `binary_search(xs: list[int], x: int) -> tuple[list[int], int | None]`: Sorts a list of elements and returns the sorted list and index of an element in the sorted list using binary search, or `None` if it is not in the list. You may treat the returned sorted list as correct and only test the returned index.
   - `merge_sort(l: list[str]) -> list[str]`: Sorts a list of strings in lexicographical order.
   - `is_subsequence(xs: list[int], ys: list[int]) -> bool`: Checks whether a list `xs` is a subsequence of `ys`. I.e. all values in `xs` must occur in the same order in `ys`, but `ys` may contain other elements in between
   - `remove_all(xs: list[int], value: int) -> list[int]`: Returns a list with all occurrences of `value` removed.
   - `dedup(xs: list[int]) -> list[int]`: Returns a list with values de-duplicated.
   - `chunked(xs: list[int], n: int) -> list[list[int]]`: Chunks a list into chunks of size `n`. If `n <= 0`, the function will raise a `ValueError`. The last chunk might be smaller than `n`, but a chunk should never be empty.
   - `parse_int(s: str) -> int`: Returns an integer parse from a string. It should ignore any whitespace in front and parse positive and negative numbers that might start with `+` or `-`. If parsing fails, the function should raise a `ValueError`.

State any preconditions you assume and enforce them via custom strategies or `assume`. In your submission add the corrected implementations and properties you created for `hypothesis` in *simple_functions_fixed.py*

2. **Generators for Property-Based Debugging** (3 points)

   You are given another set of functions in *generator_functions.py*. Again, the goal is to check whether those implementations are bug free using `hypothesis`. To properly check those implementations, default generators are usually not good enough to find suitable test cases.

   For each of the given functions:

   - Write a generator to produce useful inputs.
   - Report what kind of values your generator produces and why.
   - Write at least 2 properties that help you identify any bugs.
   - Report whether the implementation is bug free, and briefly explain why based on your properties.
   - Run `hypothesis`. If a property fails, report the minimal failing example.
   - If the implementation contains a bug, find and correct it, and check if your properties hold now.

   The file *generator_functions.py* contains the following functions:

   - `is_palindrome(s: str) -> bool`: Checks whether a string is a palindrome (= same string if read backwards). Create a custom generator that generates strings that are palindromes or near-palindromes (e.g. with one character changed).
   - `parse_csv_line(line: str) -> list[str]`: Parses a string that contains a CSV line into a list of strings for each field.

     Each field is separated by a `","` symbol, and may not contain quotation marks. If a field value contains a `","` symbol, the field value must be escaped with quotation marks. A field value may also be empty. E.g. the CSV line `test1,,"test,test"` contains three fields {`"test1"`, `""`, `"test,test"`}.

     Create a custom generator that produces valid CSV fields. To check the implementation, you first need to serialize those fields into a CSV line.
   - `normalize_path(path: str) -> str`: Normalizes a path to its canonical form (with ".", "..", and "" parts between path separators removed). "." and "" stay in the same directory. ".." move out of a directory.

     The path separator is a forward slash, "/". Absolute and relative paths should be preserved (relative path -> canonical relative path, absolute path -> canonical absolute path). Absolute paths will clamp too many ".." at the root. Relative paths starting with ".." will preserve leading "..".

Create a custom generator that produces valid paths. Generated paths should contain ".", "..", and "" path parts. The generator should produce relative and absolute paths.

- `car_price(car: Car) -> int`: Returns the price of a car based on its model and selected options.

  If the selected model or an option is not available, the function will raise a `ValueError` If the sum of all option prices is greater or equal to 5000, the total price is discounted by 500.

  Create a custom generator that produces `Car` objects. These objects should contain a valid car model and valid options. The properties should not raise `ValueErrors` during testing.

State any preconditions you assume and enforce them via custom strategies or `assume`. In your submission add the corrected implementations, generators and properties you created for `hypothesis` in *generator_functions_fixed.py*

3. **Differential Property-Based Debugging** (3 points)

   You are given two implementations of a shopping-cart pricing system based on some common types in *cart_pricer_common.py*. One implementation, *cart_pricer_correct.py*, is considered the *gold standard* (correct reference), and the other implementation, *cart_pricer.py*, may contain a bug. To make the obfuscated *cart_pricer_correct.py* more usable in an IDE, also its interface description is provided in *cart_pricer_correct.pyi*.

   Your goal is to use `hypothesis` to perform differential debugging by comparing the behavior of the buggy implementation against the gold standard. In addition, you should also write some general properties that should hold for both implementations, independent of the reference.

   - Write a generator that produces useful `Cart` objects. The generated carts should be valid and should not cause exceptions during testing.
   - Write at least 2 **differential properties** that compare the buggy implementation against the gold standard.
   - Write at least 2 **general properties** that should hold for both implementations.
   - Run `hypothesis`. If a property fails, report the minimal failing example.
   - If the buggy implementation contains a bug, fix it and check that your properties hold.

   The pricing rules are intentionally simple and use only integer numbers:

   - A cart consists of items with a `unit_price` and a `qty`.
   - The subtotal is the sum of `unit_price * qty` over all items.

- If the subtotal is greater than or equal to `BULK_THRESHOLD` (100), subtract `BULK_DISCOUNT` (15).
- If the cart uses voucher code `"VOUCHER10"`, subtract 10.
- The final total must never be negative.

In your submission, add:

- Your generators and properties in *test_cart_pricer.py*, the skeleton of this file is already provided.
- The potentially corrected buggy implementation in *cart_pricer_fixed.py*.